

F I A P

P R O M P T E N G I N E E R I N G & A R T I F I C I A L I N T E L L I G E N C E

Checkpoint 02

Prompt Toolkit

Domínio: Saúde

Membro	RM
Gabriel Augusto	567057
Leonardo Kenji	567518
Lucas Gabriel	567305
Lucas Koiti	568128
Lucas Ikeda	567616

1. Domínio e Problema

Por que Saúde?

A saúde é um dos domínios mais críticos para IA — decisões imprecisas colocam vidas em risco. O Prompt Toolkit automatiza a seleção da melhor técnica de prompting para tarefas clínicas recorrentes, como triagem de urgência, extração de dados de pacientes e sumarização de prontuários.

Problema que o toolkit resolve:

Como escolher a técnica de prompting mais eficaz para cada tipo de tarefa médica? Sem uma ferramenta comparativa, testes manuais consomem tempo e recursos. O toolkit executa as 4 técnicas automaticamente e gera métricas objetivas.

Tarefas Implementadas:

Classificação de Urgência

Triagem automática: EMERGÊNCIA, URGENTE, POUCO URGENTE, NÃO URGENTE

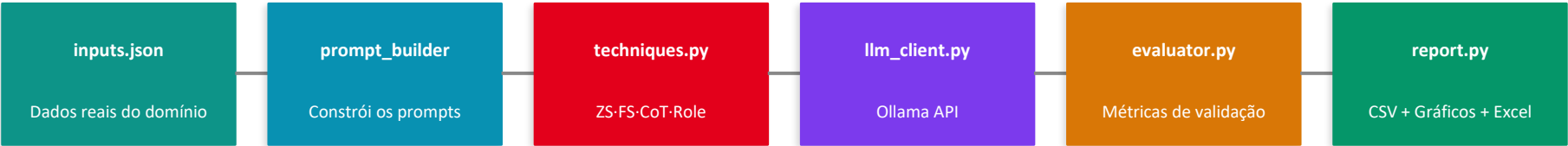
Extração de Dados do Paciente

Estruturação em JSON dos sintomas, duração e intensidade

Sumarização de Prontuário

Bullet points objetivos para o médico plantonista

2. Arquitetura do Sistema



Módulo	Responsabilidade
llm_client.py	Conexão com Ollama Cloud API, tratamento de erros e retorno de métricas
prompt_builder.py	Montagem modular do prompt: instrução, contexto, input e formato
techniques.py	Implementação das 4 técnicas: Zero-Shot, Few-Shot, CoT, Role Prompting
tasks.py	Definição das tarefas do domínio saúde com exemplos e passos de CoT
evaluator.py	Acurácia por keywords, contagem de tokens (tiktoken) e consistência
report.py	Geração de tabela CSV e 3 gráficos comparativos com matplotlib

3. Stack Técnica

Python 3.10+

Linguagem principal

tiktoken

Contagem precisa de tokens

matplotlib

Geração de gráficos comparativos

Ollama Cloud API

gpt-oss:120b — LLM gratuito

pandas

Manipulação de dados e CSV

python-dotenv

Variáveis de ambiente seguras

Como executar:

```
pip install -r requirements.txt → cp .env.example .env (adicione OLLAMA_API_KEY) → python main.py
```

4. Resultados

Tabela Comparativa de Acurácia por Técnica:

Tarefa	Zero-Shot	Few-Shot	Chain-of-Thought	Role Prompting
Classificação Urgência	80%	90%	100%	90%
Extração Dados Paciente	87%	87%	87%	87%
Sumarização Prontuário	49%	59%	52%	24%
Média Geral	72%	78,6%	79,6%	67%

Gráficos Gerados pelo Sistema:



5. Guia de Bolso — Quando Usar Cada Técnica

Técnica	Use quando...	Exemplo em Saúde	Cuidado
Zero-Shot	Tarefa objetiva com output padronizado	Classificar urgência com categorias fixas	Pode falhar em casos ambíguos
Few-Shot	Exemplos reais disponíveis, padrão deve ser seguido fielmente	Extrair dados em JSON com estrutura exata	Qualidade depende dos exemplos
Chain-of-Thought	Tarefa exige raciocínio clínico em múltiplas etapas	Triagem complexa com múltiplos sintomas	Mais tokens; mais lento
Role Prompting	Tom, vocabulário e responsabilidade profissional importam	Sumarização de prontuário com linguagem técnica	Personas genéricas reduzem o efeito

6. Reflexão

O que surpreendeu?

O que nos surpreendeu foi a proximidade das acurácias entre as técnicas. Cada uma se saiu melhor em determinada tarefa, com exceção da extração de dados do paciente, onde todas as técnicas obtiveram a mesma acurácia média de 87%. Outro ponto surpreendente foi o Chain-of-Thought atingir 100% na classificação de urgência, superando inclusive o Role Prompting, que era esperado como favorito para tarefas clínicas.

O que faríamos diferente?

- Cache para evitar chamadas repetidas à API
- Mais tarefas: laudos e tradução de terminologia
- Personas com especialidades específicas (cardiologista, pediatra)
- Interface web para melhorar a visualização do modelo em operação.
- Implementar mais métricas de validação.

Qual técnica venceu?

O Chain-of-Thought foi a técnica com maior média geral (79,6%) e venceu em duas das três tarefas — classificação de urgência com 100% e extração de dados com 87%. O Few-Shot, por sua vez, foi a melhor técnica para sumarização de prontuário com 59%, porém consumiu a maior quantidade de tokens entre todas as técnicas. O Role Prompting surpreendeu negativamente na sumarização, obtendo apenas 24% de acurácia, o pior resultado individual do projeto.

Lição aprendida

Git e GitHub foram essenciais para organizar o desenvolvimento em paralelo, com branches por módulo e commits descritivos no padrão Conventional Commits, garantindo rastreabilidade completa do projeto. O desempenho de uma técnica de prompting depende também de qual tarefa o modelo vai executar.

F I A P

Checkpoint 02 — Prompt Toolkit



Projeto Python modular com 6 módulos independentes



4 técnicas: Zero-Shot · Few-Shot · CoT · Role Prompting



3 tarefas do domínio saúde com 5 inputs reais cada



Relatório automático: tabelas CSV + 3 gráficos + recomendação



Desenvolvido com Git + GitHub (branches por módulo)